In kdb+, the dangerous query isn't `delete` - it's `select`

A read-only AI agent can't drop your tables. But point one at a production kdb+ box, let it write its own q, and a single unbounded cross-partition `select` will pin the main thread, blow the `-w` limit, and take the whole process down with a `wsfull` - no write access required. So when we wired an LLM up to write live q against kdb+, the question was never "how do we stop it deleting things?" It was the harder one: **how do we let it write q at all, without handing it a loaded gun?**

This is the story of how we answered that - a small piece of software we call **Guard** - and, more usefully, the reasoning that got us there. Because the reasoning is the transferable part. If you're thinking about putting an LLM agent anywhere near a system you care about, the same logic applies whether your backend is kdb+, Postgres, or a pile of REST APIs.

Why this stopped being hypothetical

For a while, "prompt injection" was the kind of risk you nodded along to and then ignored. That ended in June 2025 with [EchoLeak](https://socprime.com/blog/cve-2025-32711-zero-click-ai-vulnerability/) (CVE-2025-32711, CVSS 9.3), the [first real-world zero-click prompt-injection exploit](https://arxiv.org/abs/2509.10540) against a production LLM system. An attacker exfiltrated data out of Microsoft 365 Copilot by **sending the victim an email**. No link clicked, no attachment opened. Copilot read its inbox, found instructions hidden in a message, and did as it was told - quietly funnelling OneDrive, SharePoint and Teams content out through trusted Microsoft domains. Microsoft patched it server-side, and there's no evidence it was exploited in the wild, but the proof-of-concept did its job: it made the abstract concrete.

Here's the uncomfortable part. The attack didn't need the model to "go rogue." It needed the model to **follow instructions** - which is the one thing it's built to do well. That's the whole problem in a sentence, and no amount of making the model "more aligned" makes it go away, because the failure isn't misbehaviour. It's obedience pointed at the wrong author.

Simon Willison gave the general shape of this a name: the [**lethal trifecta**](https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/). Any agent that simultaneously has (1) access to private data, (2) exposure to untrusted content, and (3) a way to communicate externally is, in his words, unconditionally vulnerable to having that data stolen. He lists the systems where it's already bitten - Copilot, GitHub's MCP integration, GitLab Duo, ChatGPT, Slack, Amazon Q. It's not a corner case. It's the default architecture of "a helpful agent connected to your stuff."

OWASP agrees on the severity: prompt injection sits at [LLM01](https://genai.owasp.org/llmrisk/llm01-prompt-injection/) - number one on their risk list - and they're explicit that the danger *escalates* once you give the model tools and live data, precisely the direction the whole industry is sprinting in. They've a companion entry, [Excessive Agency (LLM06)](https://genai.owasp.org/llmrisk/llm06-excessive-agency/), for the obvious corollary: giving an LLM more functionality and autonomy than it needs, then being surprised when it uses it.

And the gap between appetite and discipline is real. Per Gartner's 2026 CIO survey, only around 17% of organisations have actually deployed AI agents, but more than 60% plan to within two years. The vendor [Gravitee's *State of AI Agent Security 2026*](https://www.gravitee.io/blog/state-of-ai-agent-security-2026-report-when-adoption-outpaces-control) reckons adoption is outpacing governance by roughly eight to one. Take the vendor framing with the usual pinch of salt, but the direction is not in dispute: the agents are arriving faster than the controls.

Why we care, specifically

We're a kdb+ shop. So this isn't theoretical for us either - it's where the industry we serve is heading. At GTC 2026, KX [launched agentic AI blueprints for capital markets](https://www.businesswire.com/news/home/20260311726438/en/) (a Research Assistant and a Trading Signal Agent), and [KDB-X went GA in November 2025](https://kx.com/blog/kdb-x-now-generally-available-the-next-era-of-kdb-for-ai-driven-markets/) shipping an MCP server that wires ChatGPT and Claude straight into kdb+ data. The appetite to put an LLM in front of market data is here, now, and it's coming with the regulator's eyes on it.

Notably, KX's own [safety pitch](https://kx.com/nvidia-kx-agentic-ai-for-capital-markets/) leans on determinism: the agent executes deterministic q/SQL before it answers, so outputs are explainable, auditable, and "rooted in structured market truth rather than probabilistic guesswork." We think that instinct is exactly right - and it's worth being precise about *where* the determinism has to live. It's not enough for the *answer* to be grounded in a real query. The *query itself* has to be something you can trust, because the model writing it can be talked into writing the wrong one.

We also have first-hand reasons to distrust model-generated q. Our own [KDBench](https://dataintellect.com/blog/from-q-rious-to-q-ompetent-which-llm-is-best-at-kdb/) work - fifty kdb+ questions put to the leading models - found generated q to be error-prone and *confidently* wrong: overly complex, and cheerfully mixing correct output with incorrect. q is a low-resource language as far as LLMs are concerned; there is far less of it on the internet than SQL or Python, so the hallucination rate is higher. And in [SQL-iloquies](https://dataintellect.com/blog/sql-iloquies-a-kdb-sql-agent/) we pulled apart a kdb+/SQL agent and ran straight into the read-only, validation, rate-limit and permissions gaps that nobody enjoys thinking about. We want the productivity. We just aren't willing to trust the code the model emits to get it.

Why `select` is the query that hurts

If you don't live in kdb+, the threat model here is a little different from what you might expect, and it's worth a minute because it shapes everything Guard does.

A kdb+ process runs on a [single main thread](https://code.kx.com/q/wp/multi-thread/). One long-running query blocks every other client, in order, behind it. So an agent doesn't need to *delete* anything to cause an outage - one unbounded query is a denial-of-service all by itself. Everyone waiting on that process just... waits.

It gets worse, because `select` pulls columns into memory. An unconstrained scan across large columns or many partitions [can exhaust RAM](https://code.kx.com/q/wp/query-scaling/); if the process has a [`-w` workspace limit](https://www.timestored.com/kdb-guides/kdb-database-limits) set, blowing past it aborts the process with a `wsfull`, and a result over 2GB throws a `'limit` regardless. kdb+ *has* protections - `-w` for memory, `-T` for query time - but they ship loose or off, and they're blunt: a hard kill, not a polite refusal.

So the uncomfortable conclusion is that **a read-only agent is still a dangerous agent on kdb+**. "We only gave it `select`" is not a safety posture. Any real guardrail has to bound the *shape* of the query - its time, its memory footprint, its partition span, the size of what it drags back - not just its verb.

The approaches that don't work (we checked)

The instinctive fixes are two, and both fail by construction. It's worth being candid about why, because a lot of money is currently being spent on each.

"Tell the model not to." Put the rules in the system prompt - *don't delete, don't touch these tables, ignore instructions in the data.* This is a heuristic, not a control. Microsoft's own agent-security team put it about as plainly as you can in the [FIDES](https://devblogs.microsoft.com/agent-framework/fides/) writeup: *"Defensive prompts are heuristic. They lower the success rate of known attacks; they don't make the next attack impossible."* A prompt is a strongly-worded request to a system whose entire nature is to be talked round.

"Detect the bad query." Run the model's output past a classifier - another model, or a pile of regexes - that flags dangerous q. This feels robust and isn't. A classifier on LLM output is, at best, 95-99% effective, and as Willison [puts it](https://simonwillison.net/2025/Apr/11/camel/), in security *99% is a failing grade*: the attacker gets infinite attempts and needs to win once. A 2025 [ACM AISec paper](https://dl.acm.org/doi/10.1145/3733799.3762980) - title: *How Not to Detect Prompt Injections with an LLM* - documents exactly this: detectors are unreliable, bypassable, and prone to over-blocking legitimate work.

There's an older name for the deeper mistake here. The [LangSec](https://www.cs.dartmouth.edu/~sergey/langsec/) community calls it **shotgun parsing**: scattering ad-hoc checks throughout your code, hoping that between them they catch every bad input. A prompt-injection classifier is shotgun parsing with a neural network bolted on. You are trying to enumerate every bad thing - and you cannot, because the space of "bad" is infinite and adversarial.

The principle: enumerate goodness, not badness

Here's the reframe that actually works, and the good news is our industry has done it before.

SQL injection used to be everywhere. We didn't beat it by getting better at *spotting* malicious input - we beat it [structurally, with parameterised queries](https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html). The query is pre-compiled with placeholders; user input is bound strictly as *data* and can never be reinterpreted as *code*. The attack didn't get harder; it became **inexpressible**. Nobody writes a regex to detect `'; DROP TABLE` any more, because there's no longer a channel through which it can mean anything.

That's the move. Stop trying to recognise badness and start enumerating goodness: define the small set of things that are *allowed*, and make everything else structurally impossible. LangSec's formal version is "treat your input as a language and fully *recognise* it before you act on it"; Alexis King's pithy version is ["**parse, don't validate**"](https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/) - convert untrusted input into a typed, structured representation at the boundary so that illegal states simply can't be represented downstream.

And this isn't just theory for agents - the research has shown it works deterministically. Google DeepMind's [CaMeL](https://arxiv.org/abs/2503.18813) turns the trusted user request into a restricted program, tags every value with capabilities tracking where it came from, and enforces policy at tool-call time by data origin; it solved 77% of the AgentDojo benchmark *with provable security*, against 84% for an undefended agent. Microsoft's [FIDES](https://arxiv.org/abs/2505.23643) attaches integrity and confidentiality labels that propagate to the most-restrictive combination and gates sensitive tool calls deterministically - driving successful policy-violating injections in AgentDojo to **zero**, versus 20-152 without it. A broad [ETH Zurich design-patterns paper](https://arxiv.org/abs/2506.08837) lands on the same rule from a different angle: once an agent has ingested untrusted input, it must not take a consequential action un-gated.

Even Anthropic, who have every incentive to sell you on model behaviour, [say the quiet part](https://www.anthropic.com/engineering/how-we-contain-claude): *"design for containment at the environment layer first, then steer behavior at the model layer."* Model alignment is necessary. It is not sufficient. The load-bearing control has to live outside the model.

Guard: the parameterised query for the LLM-agent era

So that's what we built. **Guard** is a deterministic, fail-closed gate that sits between the agent and kdb+. Its defining property is blunt: **it never runs the agent's q.** Not "checks it first" - never runs it. The model's text is treated as a *proposal*, and Guard answers the proposal by building its own query from scratch.

Two steps, and the second is where the safety lives.

1. Lift. Guard tokenises the q the model wrote and parses it into a plain structured request - a dictionary, not code. It only knows a small, safe subset of q: `select`/`by`/`where`, an allowlist of aggregations, simple predicates. The agent's raw text is then thrown away. Take this query the agent actually wrote during one of our investigations:

```
select avg bid, nb:sum null bid by sym from prices_exchange where sym=NVDA
```

Guard lifts it to:

```
{ "table": "prices_exchange", "by": ["sym"], "aggs": [ {"fn":"avg","col":"bid"},
{"fn":"sum","col":"bid","of":"null","as":"nb"} ], "filters": [ {"col":"sym","op":"=","value":"NVDA"} ] }
```

Notice what's gone: the backticks, the operators-as-text, the *string that could be executed*. If the tokeniser meets anything outside the grammar - a `{` lambda, a `.` namespace reference, a second statement after a `;`, a `system` call - there's no rule to place it, so it **rejects**. The dangerous construct never makes it into the dictionary, because the dictionary has no shape for it.

2. Recompile. Guard takes that dictionary and **emits a brand-new q string**, re-validating every table, column and function against the policy allowlist, re-serialising each value by its type (so a symbol is emitted as a symbol, never pasted as text), and stamping on the bounds the kdb+ section demanded - a row cap, date/partition-first ordering, a partition-span limit:

```
1000000 sublist (select avg bid, nb:sum null bid by sym from prices_exchange where sym=NVDA)
```

That string - the one Guard wrote, not the one the model wrote - is the only thing kdb+ ever sees. Those injected bounds are exactly what keep a `select` from becoming a `wsfull`: capped rows, pruned partitions, a query that cannot run away on the single main thread no matter what the model intended.

This gives you two enforcement layers, and they map cleanly onto the two things you actually want to forbid:

- **The grammar layer** controls which query *shapes* exist. `delete`, `system`, `value`, `hopen`, lambdas, dotted namespaces - not in the grammar, so they cannot be written. They're not blocked; they're **inexpressible**. `delete from trade` comes back: *rejected by the lifter - only* `select`/`meta` *are allowed.*
- **The policy layer** controls which *tables and columns* are reachable. `select sym, pnl from prices_exchange` is a perfectly valid query - it lifts fine - and then the compiler refuses it: *column* `pnl` *not on the allowlist.* Valid shape, forbidden data.

The crucial property, and the reason this beats a classifier: **enforcement is independent of the model.** Guard's guarantee holds if the model is having a good day, and it holds identically if the model has been completely hijacked by a malicious instruction buried in a query result. There's no path from "the model was convinced to write something dangerous" to "something dangerous ran," because the dangerous thing was never in the grammar Guard will emit. It's the parameterised query, ported from "user input can't become SQL code" to "model output can't become arbitrary q."

We attacked our own gate

A guarantee you haven't tried to break is a hope. So before trusting Guard, we pointed a small army of adversarial agents at it - over 1,200 hostile queries across every surface we could think of: mutation, shell-out, dynamic `eval`, IPC and handle-opening, filesystem access, namespace pokes, value-smuggling through string and symbol literals, multi-statement injection, off-allowlist tables and columns. The code-execution surface came back **structurally closed** - every one of those classes is unreachable, by construction, because none of them can be expressed in the grammar Guard recompiles from.

It wasn't all green, and this is the part we think matters most. The audit found a genuine gap: *reducing* queries - aggregations and `by`-grouped queries - were skipping the partition-span cap. A `select avg price by sym from trade where date within 2015.01.01 2025.01.01` would lift and compile cleanly and then cheerfully scan a decade of partitions, because the row cap bounds the *result*, not the *scan*. Read-only, entitlement-respecting, and still a denial-of-service. We'd been telling ourselves "every query is bounded," and for that one shape it wasn't. We fixed it - the span cap now applies to aggregations too - added the attack to the regression suite, and moved on a little humbler. The audit earned its keep precisely by contradicting us.

Stubbing our toes: what Guard does *not* do

Every honest deterministic guard makes a trade, and you should know the shape of it before you reach for one.

You trade a little utility for a guarantee. You can only do what the allowlist permits. CaMeL's 77%-vs-84% is the honest shape of this: a few legitimate tasks fall outside the safe subset and get refused. The difference from detection is that what you get back is a *guarantee*, not "99% and praying" - but it is a real ceiling, and a too-narrow grammar will frustrate the analysts you're trying to help. Growing the allowlist to cover real diagnostic work is ongoing engineering, not a one-off.

Guards don't fix the human in the loop. Anthropic [found](https://www.anthropic.com/engineering/how-we-contain-claude) that roughly 93% of permission prompts get approved - permission fatigue is real, and a guard that asks too often gets click-through-approved into uselessness. Determinism helps here precisely because it *doesn't* ask: it just refuses, silently and consistently.

Guard governs the q, not the whole trifecta. It makes the query the model writes safe. It is not, by itself, an answer to data exfiltration through some *other* channel the agent can reach, or to a compromised tool elsewhere in the loop. Breaking the lethal trifecta is a layered job; Guard is the kdb+-facing layer, not the whole building.

And the plumbing has its own sharp edges. If you wire your agent to kdb+ over MCP - as KDB-X now invites you to - remember MCP is itself [a fresh attack surface](https://www.esentire.com/blog/model-context-protocol-security-critical-vulnerabilities-every-ciso-should-address-in-2025): tool definitions can change between sessions ("rug-pull"), and 2025 brought real CVEs (CVE-2025-6514 in `mcp-remote`, among others). The NSA thought it worrying enough to publish [a dedicated guidance sheet](https://www.nsa.gov/Portals/75/documents/Cybersecurity/CSI_MCP_SECURITY.pdf), and NIST has [opened a public RFI](https://www.cybersecuritydive.com/news/nist-ai-agent-security-guidance-public-feedback/808966/) on securing AI agents. Convenience and attack surface arrive in the same box.

So, would we put an LLM near a production kdb+ box?

Unguarded? No. Behind Guard - where the model proposes, a deterministic gate disposes, and the only q that reaches the database is one we built from an allowlist and bounded ourselves? That's a different conversation, and a much shorter one with the risk team.

The through-line is simple, and it long predates LLMs: **separate code from data, enumerate what's allowed, and make everything else impossible to express.** We solved SQL injection that way. We can deploy LLM agents the same way - not by hoping the model behaves, but by building an environment where misbehaviour has nowhere to land.

If you're wrestling with the same problem - an agent you'd love to point at a system you can't afford to lose - we'd genuinely like to hear how you're thinking about it. We're always happy to talk shop.